Deepseek Chat

explain the code in detail:

```
# app_streamlit.py
# Streamlit dashboard for Bioreactor ML Prediction using saved sklearn Pipeline artifacts
# Works with artifacts produced by predict.py (model .joblib + feature_schema.json)

import json
import math
from pathlib import Path
import io

import numpy as np
import pandas as pd
import streamlit as st
import matplotlib.pyplot as plt
import seaborn as sns
from joblib import load, dump
from sklearn.metrics import r2_score, mean_absolute_error, mean_squared_error
from sklearn.inspection import permutation_importance
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.linear_model import RidgeCV

import streamlit_authenticator as stauth
import yaml
from yaml.loader import SafeLoader
import streamlit.components.v1 as components
import base64

# 1. MUST BE FIRST
st.set_page_config(page_title="Bioreactor ML Dashboard", layout="wide")

# 1.1 IMAGE HELPER
def get_base64_of_bin_file(bin_file):
    with open(bin_file, 'rb') as f:
        data = f.read()
    return base64.b64encode(data).decode()

def get_img_with_href(local_img_path):
    img_format = local_img_path.split(".")[-1]
    bin_str = get_base64_of_bin_file(local_img_path)
    return f"{bin_str}"
```

```
# 1.5 CUSTOM CSS DESIGN
def set_design(theme="Light", is_authenticated=False):
    bg_img = ""
    try:
        if Path("biotech_bg.png").exists():
            bg_data = get_img_with_href("biotech_bg.png")
            bg_img = f"url({bg_data})"
    except:
        pass

    # Theme configuration
    if theme == "Dark":
        bg_overlay = "rgba(16, 42, 67, 0.85)"
        text_color = "#f0f4f8"
        card_bg = "rgba(255, 255, 255, 0.1)"
        card_border = "rgba(255, 255, 255, 0.15)"
        sidebar_bg = "rgba(0, 0, 0, 0.4)"
        metric_val = "#00d1ff"
    else:
        bg_overlay = "rgba(240, 244, 248, 0.8)"
        text_color = "#102a43"
        card_bg = "rgba(255, 255, 255, 0.7)"
        card_border = "rgba(255, 255, 255, 0.5)"
        sidebar_bg = "rgba(255, 255, 255, 0.3)"
        metric_val = "#004a8c"

    # If not authenticated, we might want a cleaner background for the login
card
    if not is_authenticated:
        bg_overlay = "rgba(0, 0, 0, 0.4)" # Darken for login visibility

    st.markdown(f"""
    <style>
    @import url('https://fonts.googleapis.com/css2?
family=Inter:wght@400;600;700&display=swap');

    html, body, [class*="css"], .stMarkdown, .stText, p, label, .stCaption {{
        font-family: 'Inter', sans-serif;
        color: {text_color} !important;
    }}

    /* Specific overrides for labels and captions which often fail in Light mode
*/
    .stWidget label, .stMarkdown p, .stCaption, .stSlider label {{
        color: {text_color} !important;
        font-weight: 600 !important;
    }}

    .stApp {{
```

```
        background: {bg_overlay} {bg_img};
        background-size: cover;
        background-blend-mode: overlay;
        background-attachment: fixed;
    }}

    [data-testid="stSidebar"] {{
        background: {sidebar_bg} !important;
        backdrop-filter: blur(12px) !important;
        border-right: 1px solid {card_border} !important;
    }}

    .stExpander, div[data-testid="stMetric"], .stTabs, div[data-
testid="stDataFrame"], .stAlert {{
        background-color: {card_bg} !important;
        border-radius: 12px !important;
        border: 1px solid {card_border} !important;
        padding: 10px !important;
        box-shadow: 0 8px 32px 0 rgba(31, 38, 135, 0.07) !important;
    }}

    div[data-testid="stMetricValue"] {{
        font-weight: 700 !important;
        color: {metric_val} !important;
    }}

    h1, h2, h3, h4, h5, h6 {{
        color: {text_color} !important;
        font-weight: 700 !important;
    }}

    /* Ensure ALL buttons and their text are always visible with high contrast
*/
    .stButton > button, .stButton > button p, .stButton > button span, .stForm
submit button, .stForm submit button p {{
        color: white !important;
        font-weight: 600 !important;
    }}

    .stButton > button {{
        border-radius: 8px !important;
        border: none !important;
        background: linear-gradient(135deg, #0054A0 0%, #003a6d 100%)
!important;
        padding: 10px 24px !important;
        transition: all 0.3s cubic-bezier(0.4, 0, 0.2, 1) !important;
        width: 100% !important;
    }}
```

```
    .stButton > button:hover {{
        transform: translateY(-2px) !important;
        box-shadow: 0 4px 12px rgba(0, 84, 160, 0.3) !important;
        background: linear-gradient(135deg, #0066cc 0%, #004a8c 100%)
!important;
    }}

    /* Fix for tab labels visibility */
    .stTabs [data-baseweb="tab"] {{
        color: {text_color} !important;
    }}

    #MainMenu {{visibility: hidden;}}
    footer {{visibility: hidden;}}
    header {{background: transparent !important;}}

    .block-container {{
        padding-top: 2rem !important;
    }}

    /* Specific Login Card styling for clarity on bg */
    [data-testid="stForm"], .stForm {{
        background: rgba(255, 255, 255, 0.95) !important;
        border-radius: 15px !important;
        padding: 30px !important;
        box-shadow: 0 10px 40px rgba(0,0,0,0.3) !important;
    }}
    [data-testid="stForm"] label, .stForm label {{
        color: #102a43 !important; /* Force dark text on login form labels */
    }}
    /* Force buttons in auth forms to stay blue/white */
    .stForm button, .stForm button p {{
        background: linear-gradient(135deg, #0054A0 0%, #003a6d 100%)
!important;
        color: white !important;
    }}
    </style>
    """, unsafe_allow_html=True)

# Call design early for login page
set_design(is_authenticated=st.session_state.get("authentication_status",
False))

# 2. AUTHENTICATION SETUP
def load_config():
    with open("users.yaml") as file:
        return yaml.load(file, Loader=SafeLoader)

config = load_config()
```

```
authenticator = stauth.Authenticate(
    config['credentials'],
    config['cookie']['name'],
    config['cookie']['key'],
    config['cookie']['expiry_days']
)

# --- BRANDING ON LOGIN PAGE ---
if not st.session_state.get("authentication_status"):
    st.markdown("""
        <div style="
            text-align: center;
            padding: 50px 20px;
            margin: 40px auto;
            max-width: 800px;
            background: rgba(255, 255, 255, 0.05);
            backdrop-filter: blur(10px);
            border-radius: 24px;
            border: 1px solid rgba(255, 255, 255, 0.1);
            box-shadow: 0 20px 50px rgba(0, 0, 0, 0.3);
        ">
            <h1 style="
                font-size: 5rem;
                font-weight: 800;
                margin-bottom: 0;
                background: linear-gradient(135deg, #f0f4f8 0%, #00d1ff 100%);
                -webkit-background-clip: text;
                -webkit-text-fill-color: transparent;
                text-shadow: 0 10px 20px rgba(0,0,0,0.2);
            ">BioNexus ML</h1>
            <p style="
                font-size: 1.8rem;
                color: #d9e2ec;
                font-weight: 500;
                letter-spacing: 2px;
                text-transform: uppercase;
                margin-top: 10px;
                opacity: 0.9;
            ">Bioprocess Intelligence Dashboard</p>
            <div style="
                width: 100px;
                height: 4px;
                background: #00d1ff;
                margin: 30px auto 0;
                border-radius: 2px;
                box-shadow: 0 0 15px rgba(0, 209, 255, 0.5);
            "></div>
        </div>
```

```
    """, unsafe_allow_html=True)

st.sidebar.header("🔐 Authentication")
try:
    authenticator.login(location='sidebar')
except Exception as e:
    st.error(e)

if st.session_state.get("authentication_status"):
    username = st.session_state["username"]
    user_info = config["credentials"]["usernames"].get(username)

    if user_info:
        if not user_info.get("approved"):
            st.warning("Awaiting admin approval.")
            authenticator.logout(location='sidebar')
            st.stop()

        authenticator.logout(location='sidebar')

        # --- Dashboard Content Starts Here ---
        # Protected Dashboard Logic

        # Check if the user is an admin
        user_roles = user_info.get("roles", [])
        user_role = user_info.get("role", "")
        is_admin = "admin" in user_roles or user_role == "admin"

        if is_admin:
            with st.sidebar.expander("🔑 Admin Panel"):
                st.write("### Pending Approvals")

                # Find unapproved users (re-load config for latest)
                fresh_config = load_config()
                unapproved_users = [u for u, data in fresh_config["credentials"]
["usernames"].items() if not data.get("approved", False)]

                if not unapproved_users:
                    st.info("No users awaiting approval.")
                else:
                    for u in unapproved_users:
                        col1, col2 = st.columns([3, 1])
                        with col1:
                            st.write(f"**{u}** ({fresh_config['credentials']['usernames']
[u]['email']})")
                        with col2:
                            if st.button("Approve", key=f"approve_{u}"):
                                fresh_config["credentials"]["usernames"][u]["approved"]
= True
```

```
                        with open("users.yaml", "w") as file:
                            yaml.dump(fresh_config, file, default_flow_style=False)
                        st.success(f"Approved {u}!")
                        st.rerun()

elif st.session_state.get("authentication_status") is False:
    st.sidebar.error('Invalid credentials')
    st.stop()
elif st.session_state.get("authentication_status") is None:
    st.sidebar.warning('Please enter your username and password')

    # Add registration widget
    try:
        # Use fresh config for registration to avoid stale state
        email_of_registered_user, username_of_registered_user,
name_of_registered_user = authenticator.register_user(
            location='sidebar',
            roles=['user'],
            merge_username_email=False
        )
        if email_of_registered_user:
            # We must reload and save immediately
            current_config = load_config()
            st.sidebar.success('User registered successfully! Awaiting admin
approval.')
            current_config['credentials']['usernames']
[username_of_registered_user]['approved'] = False
            # Ensure they don't get 'admin' unless explicitly approved
            current_config['credentials']['usernames']
[username_of_registered_user]['roles'] = ['user']

            with open('users.yaml', 'w') as file:
                yaml.dump(current_config, file, default_flow_style=False)
    except Exception as e:
        st.sidebar.error(e)

    st.stop()

# 3. PROTECTED CONTENT CONTINUES
# Note: Because the entire dashboard below depends on authentication,
# we wrap it or ensure only authenticated users reach this point.
if not st.session_state.get("authentication_status"):
    st.stop()

# ---------------------------- Helpers ----------------------------
@st.cache_resource(show_spinner=False)
def load_model(model_path: str):
    clf = load(model_path)
    return clf
```

```
@st.cache_data(show_spinner=False)
def load_schema(schema_path: str) -> dict:
    with open(schema_path, 'r') as f:
        return json.load(f)

@st.cache_data(show_spinner=False)
def read_csv(uploaded_file) -> pd.DataFrame:
    if isinstance(uploaded_file, (str, Path)):
        return pd.read_csv(uploaded_file)
    else:
        return pd.read_csv(uploaded_file)

# --- Shared preprocessing (same as predict.py) ---
def preprocess(df: pd.DataFrame) -> pd.DataFrame:
    df = df.copy()
    df.columns = df.columns.str.strip()
    df = df.dropna()

    # Derived features
    if "Glucose_gL" in df.columns:
        df["Glucose_Consumption_Rate"] = df["Glucose_gL"].diff()
    if "Dissolved_Oxygen_percent" in df.columns:
        df["DO_Change"] = df["Dissolved_Oxygen_percent"].diff()
    if {"Product_Titer_gL","Cell_Viability_percent"}.issubset(df.columns):
        df["Specific_Productivity"] = df["Product_Titer_gL"] /
df["Cell_Viability_percent"]

    # Normalize agitation
    if "Agitation_RPM" in df.columns:
        scaler = StandardScaler()
        df["Agitation_Normalized"] = scaler.fit_transform(df[["Agitation_RPM"]])

    # Flags
    if "Product_Titer_gL" in df.columns:
        df["High_Titer_Flag"] = (df["Product_Titer_gL"] > 1).astype(int)
    if "Cell_Viability_percent" in df.columns:
        df["Low_Viability_Flag"] = (df["Cell_Viability_percent"] < 98).astype(int)

    return df

def align_columns(df: pd.DataFrame, schema: dict):
    feats = schema['features']
    original_cols = list(df.columns)
    # Add missing
    missing = [c for c in feats if c not in df.columns]
    for c in missing:
        df[c] = np.nan
    # Extra cols
```

```
    extra = [c for c in df.columns if c not in feats and c != schema.get('target',
'Product_Titer_gL')]
    # Reorder
    df_aligned = df[feats].copy()
    # Coerce numeric safely
    for c in df_aligned.columns:
        df_aligned.loc[:, c] = pd.to_numeric(df_aligned[c], errors='coerce')
    info = {
        'original_cols': original_cols,
        'expected_features': feats,
        'missing_features': missing,
        'extra_cols_dropped': extra,
    }
    return df_aligned, info

def compute_rmse(y_true, y_pred):
    return float(math.sqrt(mean_squared_error(y_true, y_pred)))

# ---------------------------- Helper Visuals ----------------------------
def plot_correlation_heatmap(df: pd.DataFrame):
    fig, ax = plt.subplots(figsize=(10, 8))
    # Filter for numeric columns for correlation
    corr = df.select_dtypes(include=[np.number]).corr()
    sns.heatmap(corr, annot=True, fmt=".2f", cmap='coolwarm', ax=ax,
center=0)
    ax.set_title("Feature Correlation Matrix")
    plt.tight_layout()
    return fig

def plot_time_series(df: pd.DataFrame, y_cols: list):
    if "Time_hours" not in df.columns:
        st.warning("'Time_hours' column not found for time-series.")
        return None
    fig, ax = plt.subplots(figsize=(10, 5))
    for col in y_cols:
        if col in df.columns:
            ax.plot(df["Time_hours"], df[col], marker='o', label=col, alpha=0.7)
    ax.set_title("Process Parameters over Time")
    ax.set_xlabel("Time (hours)")
    ax.set_ylabel("Value")
    ax.legend()
    ax.grid(True, linestyle='--', alpha=0.6)
    plt.tight_layout()
    return fig

def plot_feature_importance(clf, feature_names):
    # Try to get coefficients or feature importances from the pipeline
    try:
        model_step = clf.named_steps.get('model')
```

```
        if not model_step:
            # Try last step if 'model' name is not standard
            model_step = clf.steps[-1][1]

        importances = None
        title = "Feature Importance"

        if hasattr(model_step, 'coef_'):
            importances = model_step.coef_
            title = "Model Coefficients (Feature Weights)"
        elif hasattr(model_step, 'feature_importances_'):
            importances = model_step.feature_importances_
            title = "Tree-based Feature Importance"

        if importances is not None:
            # Flatten in case of multi-output or odd shapes
            importances = np.array(importances).flatten()

            # Match with feature names
            imp_df = pd.DataFrame({'Feature': feature_names, 'Importance':
importances})
            imp_df = imp_df.sort_values(by='Importance', ascending=False)

            fig, ax = plt.subplots(figsize=(8, 6))
            sns.barplot(x='Importance', y='Feature', data=imp_df,
palette='viridis', ax=ax)
            ax.set_title(title)
            plt.tight_layout()
            return fig
    except Exception:
        pass
    return None

# --- Sidebar Theme & Recorder ---
st.sidebar.markdown("---")
st.sidebar.header("🎨 Appearance")
dashboard_theme = st.sidebar.select_slider("Theme", options=["Light",
"Dark"], value="Light")
set_design(dashboard_theme, is_authenticated=True)

st.sidebar.markdown("---")
st.sidebar.header("📸 Screen Recorder")
with st.sidebar:
    components.html("""
    <script>
    let mediaRecorder;
    let recordedChunks = [];

    function startRec() {
```

```
        navigator.mediaDevices.getDisplayMedia({ video: true }).then(stream
=> {
            mediaRecorder = new MediaRecorder(stream);
            mediaRecorder.ondataavailable = e => recordedChunks.push(e.data);
            mediaRecorder.onstop = () => {
                const blob = new Blob(recordedChunks, { type: 'video/webm' });
                const url = URL.createObjectURL(blob);
                const a = document.createElement('a');
                a.href = url;
                a.download = 'bioreactor_dashboard_session.webm';
                a.click();
                recordedChunks = [];
            };
            mediaRecorder.start();
            window.parent.postMessage({type: 'rec_status', status: 'recording'},
'*');
        }).catch(err => {
            alert("Error: " + err);
        });
    }

    function stopRec() {
        if (mediaRecorder) {
            mediaRecorder.stop();
            mediaRecorder.stream.getTracks().forEach(track => track.stop());
            window.parent.postMessage({type: 'rec_status', status: 'idle'}, '*');
        }
    }
    </script>
    <div style="display: flex; gap: 10px;">
        <button onclick="startRec()" style="padding: 8px 12px; border-radius:
6px; background: #28a745; color: white; border: none; cursor: pointer; font-
weight: 600;">● Start</button>
        <button onclick="stopRec()" style="padding: 8px 12px; border-radius:
6px; background: #dc3545; color: white; border: none; cursor: pointer; font-
weight: 600;">■ Stop</button>
    </div>
    """, height=50)

# --- Dashboard Content ---
st.title("🔬 Bioreactor ML – Prediction Dashboard")

# Create Tabs
tab_predict, tab_train, tab_explore = st.tabs(["🚀 Predict & Benchmark", "🏋️
Train Model", "📊 Data Exploration"])

with tab_predict:
    st.write("### Predict results or benchmark model performance")
    st.caption("Use saved sklearn Pipeline model artifacts (.joblib) and
```

```
schema (.json) to predict Product_Titer_gL from process data.")

    st.sidebar.header("⚙️ Configuration")
    model_choice = st.sidebar.selectbox(
        "Select default model path",
        options=[
            'outputs/model_ridgecv.joblib',
            'outputs/model_gbr.joblib',
            'outputs/model_linear.joblib',
            'Custom...'
        ],
        index=0
    )

    if model_choice == 'Custom...':
        model_path = st.sidebar.text_input("Model path (.joblib)",
value='outputs/model_ridgecv.joblib')
    else:
        model_path = model_choice

    schema_path = st.sidebar.text_input("Schema path (.json)",
value='outputs/feature_schema.json')

    st.sidebar.markdown("---")
    mode = st.sidebar.radio("Mode", ["Predict (unlabeled)", "Benchmark
(labeled)"])

    st.sidebar.markdown("---")
    st.sidebar.write("**Actions**")
    if st.sidebar.button("▶️ Run"):
        st.session_state.has_run = True

    # File input
    st.subheader("📥 Input Data")
    upload = st.file_uploader("Upload CSV (new_samples or labeled data)",
type=['csv'])
    use_sample = st.checkbox("Use local sample file:
bioreactor_ml_dataset.csv (if present)")

    input_df = None
    src_desc = None
    if upload is not None:
        input_df = read_csv(upload)
        src_desc = f"Uploaded file: `{upload.name}`"
    elif use_sample and Path('bioreactor_ml_dataset.csv').exists():
        input_df = read_csv('bioreactor_ml_dataset.csv')
        src_desc = "Local sample: `bioreactor_ml_dataset.csv`"
    else:
        st.info("Upload a CSV or check 'Use local sample file' to proceed.")
```

```
    if input_df is not None:
        # Store in session state for tab_explore
        st.session_state.input_df = input_df
        with st.expander("Preview input data", expanded=False):
            st.write(src_desc)
            st.dataframe(input_df.head(20), use_container_width=True)
            st.write(f"Rows: {len(input_df):,} │ Columns:
{len(input_df.columns)}")

    # ----------------------------- Run -----------------------------
    if st.session_state.get("has_run") and input_df is not None:
        try:
            clf = load_model(model_path)
            schema = load_schema(schema_path)
            target_col = schema.get('target', 'Product_Titer_gL')

            # Preprocess
            proc_df = input_df.copy()
            proc_df = preprocess(proc_df)

            # Align columns
            X_aligned, align_info = align_columns(proc_df.copy(), schema)
            with st.expander("Schema alignment details", expanded=False):
                st.write("**Expected features:**", align_info['expected_features'])
                st.write("**Missing features auto-added (NaN):**",
align_info['missing_features'])
                st.write("**Extra columns dropped:**",
align_info['extra_cols_dropped'])

            # Prediction or Benchmark logic
            if mode == "Predict (unlabeled)":
                preds = clf.predict(X_aligned)
                out = input_df.copy()
                # Use pd.Series to auto-align based on index (handles dropped
rows as NaN)
                out['Pred_Product_Titer_gL'] = pd.Series(preds,
index=X_aligned.index)
                st.success("Predictions completed.")

                c1, c2 = st.columns([2, 1])
                with c1:
                    st.write("#### Results Preview (Top 100)")
                    st.dataframe(out.head(100), use_container_width=True)
                    csv_bytes = out.to_csv(index=False).encode('utf-8')
                    st.download_button("⬇️ Download predictions.csv",
data=csv_bytes,
                                       file_name="predictions.csv", mime="text/csv")
                with c2:
```

```
                st.write("#### Distribution")
                fig, ax = plt.subplots(figsize=(6,6))
                sns.histplot(preds, bins=20, kde=True, ax=ax, color='#0054A0')
                ax.set_title('Predicted Distribution')
                st.pyplot(fig)
            else:
                if target_col not in input_df.columns:
                    st.error(f"Target column '{target_col}' not found in provided
CSV.")
                else:
                    # Deriving from proc_df ensures alignment with X_aligned (both
processed)
                    y_series = pd.to_numeric(proc_df[target_col], errors='coerce')
                    valid_mask = y_series.notnull()
                    y_true = y_series[valid_mask]
                    X_eval = X_aligned[valid_mask]

                    if len(y_true) == 0:
                        st.error("No valid numeric target values found in the target
column after preprocessing.")
                    else:
                        preds = clf.predict(X_eval)
                        r2 = r2_score(y_true, preds)
                        mae = mean_absolute_error(y_true, preds)
                        rmse = compute_rmse(y_true, preds)

                        st.write("#### 📊 Performance Metrics")
                        c1, c2, c3 = st.columns(3)
                        c1.metric("Test R²", f"{r2:.4f}")
                        c2.metric("MAE (g/L)", f"{mae:.4f}")
                        c3.metric("RMSE (g/L)", f"{rmse:.4f}")

                        col_a, col_b = st.columns(2)
                        with col_a:
                            st.write("#### 🎯 Actual vs Predicted")
                            fig, ax = plt.subplots(figsize=(6,6))
                            ax.scatter(y_true, preds, alpha=0.6, color='darkblue')
                            lims = [min(y_true.min(), preds.min()), max(y_true.max(),
preds.max())]
                            ax.plot(lims, lims, 'r--', label='Ideal')
                            ax.set_xlabel('Actual')
                            ax.set_ylabel('Predicted')
                            ax.legend()
                            st.pyplot(fig)

                        with col_b:
                            st.write("#### 💡 Model Insights")
                            feat_fig = plot_feature_importance(clf, schema['features'])
                            if feat_fig:
```

```
                    st.pyplot(feat_fig)
                else:
                    st.info("Feature importance not available for this model
type.")

            with st.expander("Residuals Analysis"):
                residuals = y_true - preds
                fig, ax = plt.subplots(figsize=(10,4))
                sns.histplot(residuals, bins=30, kde=True, ax=ax,
color='#00A1DE')
                ax.axvline(0, color='red', linestyle='--')
                ax.set_title('Residuals Distribution (Actual - Predicted)')
                st.pyplot(fig)

    except Exception as e:
        st.exception(e)

with tab_train:
    st.write("### 🏋️ Train a New Model")
    st.caption("Upload labeled data to train a new RidgeCV pipeline. This will
save a `.joblib` model and a `.json` schema.")

    train_upload = st.file_uploader("Upload Training CSV", type=['csv'],
key="train_up")

    c1, c2 = st.columns(2)
    with c1:
        target_name = st.text_input("Target Column", value="Product_Titer_gL")
        out_model_name = st.text_input("Output Model Path",
value="outputs/model_custom.joblib")
    with c2:
        out_schema_name = st.text_input("Output Schema Path",
value="outputs/feature_schema_custom.json")

    if st.button("🔥 Start Training"):
        if train_upload is not None:
            try:
                with st.spinner("Preprocessing and training..."):
                    raw_train_df = pd.read_csv(train_upload)
                    if target_name not in raw_train_df.columns:
                        st.error(f"Target '{target_name}' not found.")
                    else:
                        # 1. Preprocess
                        df_processed = preprocess(raw_train_df)

                        # 2. Train
                        X_train = df_processed.drop(columns=[target_name])
                        y_train = df_processed[target_name]
```

```
                pipe = Pipeline([
                    ('imputer', SimpleImputer(strategy='median')),
                    ('scaler', StandardScaler()),
                    ('model', RidgeCV(alphas=np.logspace(-3, 3, 13), cv=5))
                ])
                pipe.fit(X_train, y_train)

                # 3. Save
                Path("outputs").mkdir(exist_ok=True)
                dump(pipe, out_model_name)

                schema = {
                    'target': target_name,
                    'features': X_train.columns.tolist(),
                    'dtypes': {c: str(df_processed[c].dtype) for c in
X_train.columns},
                    'trained_rows': int(df_processed.shape[0]),
                    'trained_cols': int(df_processed.shape[1])
                }
                with open(out_schema_name, 'w') as f:
                    json.dump(schema, f, indent=2)

                st.success(f"✅ Model trained and saved to
`{out_model_name}`")

                # 4. Results
                train_preds = pipe.predict(X_train)
                r2_train = r2_score(y_train, train_preds)
                st.metric("Training R²", f"{r2_train:.4f}")
            except Exception as e:
                st.exception(e)
        else:
            st.warning("Please upload a training dataset first.")

with tab_explore:
    explore_df = st.session_state.get("input_df")

    if explore_df is None:
        st.info("Please upload data in the 'Predict' tab to begin exploration.")
    else:
        st.write("### 🔍 Process Data Exploration")

        m1, m2, m3, m4 = st.columns(4)
        if "Temperature_C" in explore_df.columns:
            m1.metric("Avg Temp", f"{explore_df['Temperature_C'].mean():.1f}
°C")
        if "pH" in explore_df.columns:
            m2.metric("Avg pH", f"{explore_df['pH'].mean():.2f}")
        if "Dissolved_Oxygen_percent" in explore_df.columns:
```

```
        m3.metric("Avg DO", f"
{explore_df['Dissolved_Oxygen_percent'].mean():.1f} %")
    if "Product_Titer_gL" in explore_df.columns:
        m4.metric("Avg Titer", f"{explore_df['Product_Titer_gL'].mean():.2f}
g/L")

    st.markdown("---")
    col1, col2 = st.columns(2)

    with col1:
        st.write("#### 🔥 Parameter Correlations")
        heat_fig = plot_correlation_heatmap(explore_df)
        st.pyplot(heat_fig)

    with col2:
        st.write("#### 📈 Time-Series Trends")
        all_cols = [c for c in explore_df.columns if c != "Time_hours"]
        selected_metrics = st.multiselect(
            "Select parameters",
            options=all_cols,
            default=["Temperature_C", "pH"] if "Temperature_C" in all_cols and
"pH" in explore_df.columns else all_cols[:2]
        )
        if selected_metrics:
            ts_fig = plot_time_series(explore_df, selected_metrics)
            if ts_fig:
                st.pyplot(ts_fig)

    st.markdown("---")
    st.write("#### 📊 Distributions")
    numeric_cols = explore_df.select_dtypes(include=
[np.number]).columns
    dist_col = st.selectbox("Select feature", options=numeric_cols,
key="dist_sel")

    c_hist, c_box = st.columns([2, 1])
    with c_hist:
        fig, ax = plt.subplots(figsize=(8, 4))
        sns.histplot(explore_df[dist_col], kde=True, ax=ax, color='teal')
        st.pyplot(fig)
    with c_box:
        fig, ax = plt.subplots(figsize=(4, 4))
        sns.boxplot(y=explore_df[dist_col], ax=ax, color='lightgreen')
        st.pyplot(fig)
```

This is a comprehensive Streamlit dashboard application for bioreactor machine learning predictions. Let me break it down in detail:

Overview

This dashboard allows users to load trained ML models (scikit-learn pipelines) and make predictions on bioreactor process data, with features for model training, benchmarking, and data exploration.

Key Components

1. Authentication System (Lines 55-185)

- Uses `streamlit_authenticator` for user management
- Reads user credentials from `users.yaml` file
- Features:
  - User registration with pending approval
  - Role-based access (admin vs regular users)
  - Admin panel for approving new users
  - Cookie-based session management

2. UI/UX Design (Lines 22-83)

- Custom CSS styling with theme support (Light/Dark)
- Background image support with overlay effects
- Responsive design with modern glass-morphism effects
- Font imports (Inter font family)
- Consistent color scheme and component styling

3. Core Functionality Tabs

Tab 1: Predict & Benchmark (Lines 284-448)

- **Model Loading**: Loads pre-trained sklearn pipelines (.joblib files)
- **Schema Management**: Uses feature_schema.json to validate input data
- **Two Modes**:
  - **Prediction Mode**: Makes predictions on unlabeled data
  - **Benchmark Mode**: Compares predictions with actual values
- **Features**:
  - Data preprocessing (same as predict.py)
  - Column alignment with schema
  - Performance metrics (R^2, MAE, RMSE)

- Visualization of results (distribution plots, actual vs predicted)
- Residual analysis
- Model coefficient/importance plots

Tab 2: Train Model (Lines 450-494)

- Allows training new RidgeCV models
- Features:
  - Upload training data (CSV)
  - Specify target column
  - Automatic preprocessing
  - Pipeline creation (imputer → scaler → model)
  - Save trained model and schema
  - Display training R^2 score

Tab 3: Data Exploration (Lines 496-538)

- Exploratory data analysis tools:
  - Summary statistics (mean values)
  - Correlation heatmaps
  - Time-series plotting (if Time_hours column exists)
  - Distribution analysis (histograms + box plots)
  - Interactive feature selection

4. Data Preprocessing Pipeline (Lines 170-199)

- Feature engineering:
  - Glucose consumption rate calculation
  - Dissolved oxygen change
  - Specific productivity (titer/viability)
  - Normalized agitation (StandardScaler)
  - Binary flags (high titer, low viability)
- Handles missing values and data alignment

5. Visualization Helpers (Lines 210-252)

- Correlation heatmaps
- Time-series plots
- Feature importance visualization
- Distribution plots

- Residual analysis

6. Additional Features

- **Screen Recorder** (Lines 267-282): Embedded JavaScript for session recording
- **Theme Switcher** (Lines 265-266): Light/Dark mode toggle
- **File Export**: Download predictions as CSV
- **Caching**: Uses Streamlit's caching for model loading and data processing

Data Flow

1. **Authentication** → User logs in/registers
2. **Data Input** → Upload CSV or use sample data
3. **Preprocessing** → Feature engineering + alignment with schema
4. **Prediction** → Load model → Make predictions → Display results
5. **Export** → Download results or visualize insights

File Structure Dependencies

- `users.yaml`: User credentials and roles
- `outputs/model_*.joblib`: Pre-trained models
- `outputs/feature_schema.json`: Feature schema for validation
- `bioreactor_ml_dataset.csv`: Sample dataset (optional)
- `biotech_bg.png`: Background image (optional)

Key Libraries Used

- **streamlit**: Web app framework
- **scikit-learn**: ML models and preprocessing
- **pandas/numpy**: Data manipulation
- **matplotlib/seaborn**: Visualizations
- **joblib**: Model serialization
- **streamlit_authenticator**: Authentication
- **PyYAML**: YAML file handling

This dashboard provides a complete MLOps workflow for bioreactor predictions, from data exploration to model training and deployment, with enterprise-grade features like authentication and user management.

explain more

Deep Dive: Bioreactor ML Dashboard Architecture

Let me provide a more comprehensive technical explanation of this sophisticated Streamlit application.

1. Authentication System Deep Dive (Lines 55-185)

User Management Architecture

```python
# users.yaml structure
credentials:
  usernames:
    john_doe:
      email: john@company.com
      name: John Doe
      password: hashed_password  # bcrypt hashed
      approved: true/false
      roles: ['admin', 'user']
      role: admin  # legacy field
```

The authentication system implements a multi-tier access control:

1. **Registration Flow**:
   - New users register via sidebar form
   - `authenticator.register_user()` creates pending account
   - `approved: False` flag set by default
   - User data written back to YAML file
2. **Approval Workflow**:
   - Admin users see pending approvals in sidebar
   - One-click approval updates YAML file
   - `st.rerun()` refreshes UI with new permissions
3. **Session Management**:
   - Encrypted cookies for persistent login
   - Configurable expiry days
   - Role-based access control (RBAC)

2. Model Pipeline Architecture

Pipeline Components

The application works with scikit-learn pipelines that typically include:

```python
Pipeline([
    ('imputer', SimpleImputer(strategy='median')),
    ('scaler', StandardScaler()),
    ('model', RidgeCV(alphas=np.logspace(-3, 3, 13), cv=5))
])
```

Schema File Structure (`feature_schema.json`)

```json
{
    "target": "Product_Titer_gL",
    "features": ["Temperature_C", "pH", "Glucose_gL", "Dissolved_Oxygen_percent",
...],
    "dtypes": {
        "Temperature_C": "float64",
        "pH": "float64",
        "Glucose_gL": "float64"
    },
    "trained_rows": 1000,
    "trained_cols": 15
}
```

3. Advanced Data Preprocessing (Lines 170-199)

Feature Engineering Logic

The preprocessing function creates domain-specific features:

```python
def preprocess(df: pd.DataFrame) -> pd.DataFrame:
    # 1. Rate Calculations
    df["Glucose_Consumption_Rate"] = df["Glucose_gL"].diff()
    # ΔGlucose/Δt - metabolic activity indicator

    # 2. Delta Measurements
    df["DO_Change"] = df["Dissolved_Oxygen_percent"].diff()
    # Oxygen uptake rate

    # 3. Derived Metrics
    df["Specific_Productivity"] = df["Product_Titer_gL"] / df["Cell_Viability_per
cent"]
    # Productivity per viable cell
```

```
    # 4. Normalized Features
    scaler = StandardScaler()
    df["Agitation_Normalized"] = scaler.fit_transform(df[["Agitation_RPM"]])
    # Standardized for model stability

    # 5. Binary Indicators
    df["High_Titer_Flag"] = (df["Product_Titer_gL"] > 1).astype(int)
    df["Low_Viability_Flag"] = (df["Cell_Viability_percent"] < 98).astype(int)
```

Column Alignment Strategy (Lines 201-218)

The `align_columns()` function handles train-inference skew:

```python
def align_columns(df: pd.DataFrame, schema: dict):
    # Problem: Model expects specific features in specific order
    # Solution: Reorder and pad missing columns

    # Missing columns → fill with NaN
    missing = [c for c in feats if c not in df.columns]
    for c in missing:
        df[c] = np.nan  # Model must handle missing values

    # Extra columns → drop silently
    extra = [c for c in df.columns if c not in feats]

    # Reorder to match training data exactly
    df_aligned = df[feats].copy()
```

4. Visualization System Deep Dive

Custom Styling Engine (Lines 22-83)

The `set_design()` function implements dynamic theming:

```python
def set_design(theme="Light", is_authenticated=False):
    # Dynamic CSS generation based on:
    # - Theme selection (Light/Dark)
    # - Authentication status (login screen vs dashboard)
    # - Background image availability

    # CSS Variables:
    # - bg_overlay: semi-transparent overlay for readability
    # - text_color: contrast-optimized for theme
    # - card_bg: glass-morphism effect with backdrop-filter
    # - metric_val: accent color for metrics
```

Plotting Architecture

Multiple visualization types with consistent styling:

1. **Correlation Matrix** (Lines 210-217)
   - Uses seaborn heatmap
   - Automatically filters numeric columns
   - Annotated with correlation coefficients
2. **Time Series** (Lines 218-230)
   - Requires 'Time_hours' column
   - Multi-trace plotting
   - Automatic legend and grid
3. **Feature Importance** (Lines 231-252)
   - Polymorphic: handles both coefficients and tree-based importances
   - Flattens multi-output models
   - Sorted bar charts

5. State Management

Session State Usage

```python
# Key session state variables:
st.session_state.update({
    "authentication_status": True/False/None,
    "username": "current_user",
    "has_run": False,  # Trigger prediction
    "input_df": None   # Cached input data
})
```

Caching Strategy

```python
@st.cache_resource(show_spinner=False)
def load_model(model_path: str):
    # Caches model in memory across reruns
    # Resource cache for large objects

@st.cache_data(show_spinner=False)
def load_schema(schema_path: str) -> dict:
    # Caches JSON schema
    # Data cache for serializable objects
```

6. Error Handling & Edge Cases

Robust Prediction Pipeline

```python
try:
    # 1. Model loading with fallback
    clf = load_model(model_path)

    # 2. Data validation
    if target_col not in input_df.columns and mode == "Benchmark":
        st.error("Target column missing")

    # 3. Numeric coercion with error handling
    df_aligned.loc[:, c] = pd.to_numeric(df_aligned[c], errors='coerce')

    # 4. Mask invalid values
    valid_mask = y_series.notnull()

    # 5. Graceful degradation for feature importance
    if feat_fig is None:
        st.info("Feature importance not available")
except Exception as e:
    st.exception(e)  # Full traceback for debugging
```

7. Performance Optimizations

Memory Management

- Chunked dataframe previews (head(100))
- Lazy loading of visualizations
- Proper cleanup of matplotlib figures

Computational Efficiency

```python
# Vectorized operations throughout
preds = clf.predict(X_aligned)  # Batch prediction
out['Pred_Product_Titer_gL'] = pd.Series(preds, index=X_aligned.index)  # Aligned
assignment

# Efficient statistical computations
r2 = r2_score(y_true, preds)  # Optimized sklearn metrics
```

8. Security Considerations

Input Validation

- YAML file safe loading with `SafeLoader`
- Path traversal prevention (Path objects)
- File type restrictions (CSV only)

Authentication Security

- Password hashing (bcrypt via streamlit-authenticator)
- Encrypted cookies
- Session expiry
- Admin approval requirement

9. Extensibility Points

The code is designed for easy extension:

1. **New Models**: Simply add to model_choice dropdown
2. **New Features**: Extend preprocess() function
3. **New Visualizations**: Add functions to visualization helpers
4. **New Tabs**: Follow tab_* pattern
5. **Custom Themes**: Extend set_design() CSS

10. Deployment Considerations

Production Requirements

```bash
# Required files
- app_streamlit.py
- users.yaml (with proper permissions)
- outputs/ (directory with models)
- requirements.txt

# Optional assets
- biotech_bg.png
- bioreactor_ml_dataset.csv

# Run command
streamlit run app_streamlit.py --server.port 8501 --server.address 0.0.0.0
```

Scaling Considerations

- Use Redis for session state in multi-user deployments
- Implement database backend instead of YAML
- Add model versioning
- Implement API endpoints for programmatic access

This architecture demonstrates production-ready practices: authentication, state management, error handling, performance optimization, and extensibility - all within a single-file Streamlit application.

create pdf

I'll help you create a PDF document from this code explanation. Since I cannot directly create files, I'll provide you with the complete markdown content that you can convert to PDF using various tools (like Pandoc, Typora, or online converters).

```markdown
# Bioreactor ML Dashboard - Technical Documentation

## Complete Code Analysis and Architecture Guide

---

## Table of Contents
1. [Executive Summary](#executive-summary)
2. [System Architecture](#system-architecture)
3. [Authentication System](#authentication-system)
4. [Core Functionality](#core-functionality)
5. [Data Processing Pipeline](#data-processing-pipeline)
6. [Visualization System](#visualization-system)
7. [State Management](#state-management)
8. [Security Features](#security-features)
9. [Performance Optimizations](#performance-optimizations)
10. [Deployment Guide](#deployment-guide)
11. [Code Reference](#code-reference)
12. [Appendices](#appendices)

---

## Executive Summary

### Application Overview
The Bioreactor ML Dashboard is a comprehensive Streamlit-based web application de
signed for bioprocess monitoring and prediction using machine learning models. It
provides an end-to-end workflow for:

- **Model Deployment**: Load and use pre-trained scikit-learn pipelines
- **Prediction Services**: Real-time predictions on bioreactor process data
- **Model Training**: Train new RidgeCV models with custom data
```

```
- **Data Exploration**: Comprehensive EDA tools for process parameters
- **User Management**: Role-based access control with approval workflow
- **Performance Benchmarking**: Compare model predictions with actual values

### Key Technologies
- **Frontend**: Streamlit 1.28+
- **ML Framework**: scikit-learn 1.3+
- **Data Processing**: pandas, numpy
- **Visualization**: matplotlib, seaborn
- **Authentication**: streamlit-authenticator
- **Configuration**: PyYAML, JSON

---

## System Architecture

### High-Level Architecture Diagram
```

```
┌─────────────────────────────────────────────────────────────────────────────┐
│ Streamlit Application │
├─────────────────────────────────────────────────────────────────────────────┤
│ │
│ ┌──────────────────┐ ┌──────────────────┐ ┌──────────────────┐ │
│ │ Predict │ │ Train │ │ Explore │ │
│ │ Tab │ │ Tab │ │ Tab │ │
│ └──────────────────┘ └──────────────────┘ └──────────────────┘ │
│ │
│
┌─────────────────────────────────────────────────────────────────────────────┐
│ │ Authentication Layer │ │
│
└─────────────────────────────────────────────────────────────────────────────┘ │
│ │
│
┌─────────────────────────────────────────────────────────────────────────────┐
│ │ State Management │ │
│
└─────────────────────────────────────────────────────────────────────────────┘ │
│ │
├─────────────────────────────────────────────────────────────────────────────
```

```
──────────┤
│ External Dependencies │
├──────────────
```

```
credentials:
  usernames:
    admin_user:
      email: admin@bioreactor.com
      name: Admin User
      password: $2b$12$hashed_password_here  # bcrypt hashed
      approved: true
      roles:
      - admin
      - user
    pending_user:
      email: newuser@company.com
      name: New User
      password: $2b$12$hashed_password_here
      approved: false
      roles:
      - user
```

Authentication Flow

1. Login Process

```python
# Authentication initialization
authenticator = stauth.Authenticate(
    config['credentials'],
    config['cookie']['name'],
    config['cookie']['key'],
    config['cookie']['expiry_days']
)

# Login widget placement
authenticator.login(location='sidebar')
```

2. Session State Management

```python
# Authentication status tracking
st.session_state.get("authentication_status", False)

# User information storage
username = st.session_state["username"]
user_info = config["credentials"]["usernames"].get(username)
```

3. Approval Workflow

```python
```

```
# Admin panel for user approval
unapproved_users = [u for u, data in fresh_config["credentials"]["usernames"].ite
ms()
                    if not data.get("approved", False)]

for u in unapproved_users:
    if st.button("Approve", key=f"approve_{u}"):
        fresh_config["credentials"]["usernames"][u]["approved"] = True
        # Persist changes
        with open("users.yaml", "w") as file:
            yaml.dump(fresh_config, file)
```

4. Registration Flow

```python
# New user registration
email_of_registered_user, username_of_registered_user, name_of_registered_user =
\
    authenticator.register_user(
        location='sidebar',
        roles=['user'],
        merge_username_email=False
    )

# Post-registration approval flag
current_config['credentials']['usernames'][username_of_registered_user]['approve
d'] = False
```

Role-Based Access Control (RBAC)

Role	Permissions	Features
Admin	Full access	Approve users, all dashboard features, admin panel
User	Limited access	Predictions, exploration, training (read-only)
Pending	No access	Registration only

Core Functionality

Tab 1: Predict & Benchmark

Model Loading System

```python
@st.cache_resource(show_spinner=False)
def load_model(model_path: str):
    """Cached model loading for performance"""
    clf = load(model_path)
    return clf

@st.cache_data(show_spinner=False)
def load_schema(schema_path: str) -> dict:
    """Cached schema loading"""
    with open(schema_path, 'r') as f:
        return json.load(f)
```

Supported Model Types

```python
model_choice = st.sidebar.selectbox(
    "Select default model path",
    options=[
        'outputs/model_ridgecv.joblib',  # Ridge Regression
        'outputs/model_gbr.joblib',      # Gradient Boosting
        'outputs/model_linear.joblib',    # Linear Regression
        'Custom...'
    ]
)
```

Prediction Mode Workflow

```python
if mode == "Predict (unlabeled)":
    # 1. Generate predictions
    preds = clf.predict(X_aligned)

    # 2. Align with original data
    out = input_df.copy()
    out['Pred_Product_Titer_gL'] = pd.Series(preds, index=X_aligned.index)

    # 3. Display results
    st.dataframe(out.head(100))

    # 4. Distribution analysis
    fig, ax = plt.subplots()
    sns.histplot(preds, bins=20, kde=True, ax=ax)

    # 5. Export functionality
    csv_bytes = out.to_csv(index=False).encode('utf-8')
    st.download_button("⬇️ Download predictions.csv", data=csv_bytes)
```

Benchmark Mode Metrics

```python
# Performance calculations
r2 = r2_score(y_true, preds)                    # R² score
mae = mean_absolute_error(y_true, preds)        # Mean Absolute Error
rmse = compute_rmse(y_true, preds)              # Root Mean Square Error

def compute_rmse(y_true, y_pred):
    return float(math.sqrt(mean_squared_error(y_true, y_pred)))
```

Tab 2: Train Model

Training Pipeline Construction

```python
# Complete ML pipeline
pipe = Pipeline([
    ('imputer', SimpleImputer(strategy='median')),  # Handle missing values
    ('scaler', StandardScaler()),                    # Feature scaling
    ('model', RidgeCV(alphas=np.logspace(-3, 3, 13), cv=5))  # Cross-validated mo
del
])

# Training execution
pipe.fit(X_train, y_train)
```

Model Persistence

```python
# Save model
dump(pipe, out_model_name)

# Save schema
schema = {
    'target': target_name,
    'features': X_train.columns.tolist(),
    'dtypes': {c: str(df_processed[c].dtype) for c in X_train.columns},
    'trained_rows': int(df_processed.shape[0]),
    'trained_cols': int(df_processed.shape[1])
}

with open(out_schema_name, 'w') as f:
    json.dump(schema, f, indent=2)
```

Tab 3: Data Exploration

Statistical Summaries

```python
# Key metrics display
if "Temperature_C" in explore_df.columns:
    m1.metric("Avg Temp", f"{explore_df['Temperature_C'].mean():.1f} °C")
if "pH" in explore_df.columns:
    m2.metric("Avg pH", f"{explore_df['pH'].mean():.2f}")
```

Visualization Functions

```python
def plot_correlation_heatmap(df: pd.DataFrame):
    """Generate correlation matrix heatmap"""
    corr = df.select_dtypes(include=[np.number]).corr()
    sns.heatmap(corr, annot=True, fmt=".2f", cmap='coolwarm')

def plot_time_series(df: pd.DataFrame, y_cols: list):
    """Plot multiple parameters over time"""
    for col in y_cols:
        ax.plot(df["Time_hours"], df[col], marker='o', label=col)
```

Data Processing Pipeline

Preprocessing Functions

1. Rate Calculations

```python
# Glucose consumption rate (metabolic activity)
df["Glucose_Consumption_Rate"] = df["Glucose_gL"].diff()

# Dissolved oxygen change (oxygen uptake rate)
df["DO_Change"] = df["Dissolved_Oxygen_percent"].diff()
```

2. Derived Metrics

```python
# Specific productivity (per viable cell)
df["Specific_Productivity"] = df["Product_Titer_gL"] / df["Cell_Viability_percen
t"]
```

3. Normalization

```python
```

```
# Standardize agitation values
scaler = StandardScaler()
df["Agitation_Normalized"] = scaler.fit_transform(df[["Agitation_RPM"]])
```

4. Binary Flags

```python
# High titer indicator (>1 g/L)
df["High_Titer_Flag"] = (df["Product_Titer_gL"] > 1).astype(int)

# Low viability warning (<98%)
df["Low_Viability_Flag"] = (df["Cell_Viability_percent"] < 98).astype(int)
```

Column Alignment System

Handling Feature Mismatch

```python
def align_columns(df: pd.DataFrame, schema: dict):
    """
    Align input data with model's expected features
    Handles missing columns and extra columns gracefully
    """
    feats = schema['features']

    # Add missing columns as NaN
    missing = [c for c in feats if c not in df.columns]
    for c in missing:
        df[c] = np.nan

    # Identify extra columns to drop
    extra = [c for c in df.columns if c not in feats]

    # Reorder columns to match training data
    df_aligned = df[feats].copy()

    # Ensure numeric dtype
    for c in df_aligned.columns:
        df_aligned.loc[:, c] = pd.to_numeric(df_aligned[c], errors='coerce')

    return df_aligned, {
        'original_cols': original_cols,
        'expected_features': feats,
        'missing_features': missing,
        'extra_cols_dropped': extra
    }
```

Data Validation

Type Coercion

```python
# Safe numeric conversion with error handling
pd.to_numeric(df_aligned[c], errors='coerce')
```

Missing Value Handling

```python
# Create mask for valid values
valid_mask = y_series.notnull()
y_true = y_series[valid_mask]
X_eval = X_aligned[valid_mask]
```

Visualization System

Custom CSS Styling

Theme Engine

```python
def set_design(theme="Light", is_authenticated=False):
    """
    Dynamic CSS generation based on:
    - Theme selection (Light/Dark)
    - Authentication status
    - Background image availability
    """

    if theme == "Dark":
        bg_overlay = "rgba(16, 42, 67, 0.85)"
        text_color = "#f0f4f8"
        card_bg = "rgba(255, 255, 255, 0.1)"
        metric_val = "#00d1ff"
    else:
        bg_overlay = "rgba(240, 244, 248, 0.8)"
        text_color = "#102a43"
        card_bg = "rgba(255, 255, 255, 0.7)"
        metric_val = "#004a8c"
```

Glass Morphism Effects

```css
.stExpander, div[data-testid="stMetric"], .stTabs {
    background-color: {card_bg} !important;
```

```
    border-radius: 12px !important;
    border: 1px solid {card_border} !important;
    padding: 10px !important;
    box-shadow: 0 8px 32px 0 rgba(31, 38, 135, 0.07) !important;
    backdrop-filter: blur(12px) !important;
}
```

Visualization Types

1. Correlation Heatmap

```python
def plot_correlation_heatmap(df: pd.DataFrame):
    fig, ax = plt.subplots(figsize=(10, 8))
    corr = df.select_dtypes(include=[np.number]).corr()
    sns.heatmap(corr, annot=True, fmt=".2f", cmap='coolwarm', ax=ax, center=0)
    ax.set_title("Feature Correlation Matrix")
    return fig
```

2. Time Series Plot

```python
def plot_time_series(df: pd.DataFrame, y_cols: list):
    fig, ax = plt.subplots(figsize=(10, 5))
    for col in y_cols:
        if col in df.columns:
            ax.plot(df["Time_hours"], df[col], marker='o', label=col, alpha=0.7)
    ax.set_title("Process Parameters over Time")
    ax.set_xlabel("Time (hours)")
    ax.legend()
    ax.grid(True, linestyle='--', alpha=0.6)
    return fig
```

3. Feature Importance

```python
def plot_feature_importance(clf, feature_names):
    """Extract and plot feature importance from various model types"""
    model_step = clf.named_steps.get('model') or clf.steps[-1][1]

    if hasattr(model_step, 'coef_'):
        importances = model_step.coef_.flatten()
        title = "Model Coefficients"
    elif hasattr(model_step, 'feature_importances_'):
        importances = model_step.feature_importances_
        title = "Feature Importance"

    imp_df = pd.DataFrame({'Feature': feature_names, 'Importance': importances})
```

```
    imp_df = imp_df.sort_values(by='Importance', ascending=False)

    fig, ax = plt.subplots(figsize=(8, 6))
    sns.barplot(x='Importance', y='Feature', data=imp_df, palette='viridis', ax=a
x)
    ax.set_title(title)
    return fig
```

4. Residual Analysis

```python
residuals = y_true - preds
fig, ax = plt.subplots(figsize=(10,4))
sns.histplot(residuals, bins=30, kde=True, ax=ax, color='#00A1DE')
ax.axvline(0, color='red', linestyle='--')
ax.set_title('Residuals Distribution (Actual - Predicted)')
```

5. Distribution Analysis

```python
# Histogram with KDE
fig, ax = plt.subplots(figsize=(8, 4))
sns.histplot(explore_df[dist_col], kde=True, ax=ax, color='teal')

# Box plot for outliers
fig, ax = plt.subplots(figsize=(4, 4))
sns.boxplot(y=explore_df[dist_col], ax=ax, color='lightgreen')
```

State Management

Session State Architecture

Key State Variables

```python
# Authentication state
st.session_state["authentication_status"]  # True/False/None
st.session_state["username"]                # Current user

# Application state
st.session_state["has_run"]                 # Prediction trigger
st.session_state["input_df"]                 # Cached input data
```

State Persistence

```python
# Initialize state if not exists
if "has_run" not in st.session_state:
    st.session_state["has_run"] = False

# Update state
st.session_state["has_run"] = True

# Access state
if st.session_state.get("has_run") and input_df is not None:
    # Execute prediction
```

Caching Strategy

Resource Cache (Models)

```python
@st.cache_resource(show_spinner=False)
def load_model(model_path: str):
    """
    Cache model in memory
    - Suitable for large objects
    - Persists across reruns
    - Global singleton pattern
    """
    clf = load(model_path)
    return clf
```

Data Cache (Schema, DataFrames)

```python
@st.cache_data(show_spinner=False)
def load_schema(schema_path: str) -> dict:
    """
    Cache serializable data
    - Hash-based invalidation
    - Suitable for JSON, DataFrames
    """
    with open(schema_path, 'r') as f:
        return json.load(f)

@st.cache_data(show_spinner=False)
def read_csv(uploaded_file) -> pd.DataFrame:
    """
    Cache uploaded data
    - Prevents reloading on rerun
    """
    return pd.read_csv(uploaded_file)
```

Security Features

Authentication Security

Password Hashing

```python
# streamlit-authenticator uses bcrypt
# Passwords stored as bcrypt hashes in users.yaml
password: $2b$12$EixZaYVK1fsbw1ZfbX3OXePaWxn96p36JRZZ3yC9XqK9H7F7qYJ8O
```

Session Cookies

```python
config = {
    'cookie': {
        'name': 'bioreactor_cookie',
        'key': 'secure_random_key_here',
        'expiry_days': 30
    }
}
```

Input Validation

File Upload Safety

```python
# Restrict file types
upload = st.file_uploader("Upload CSV", type=['csv'])

# Safe path handling
from pathlib import Path
if Path("biotech_bg.png").exists():
    # Safe file existence check
```

YAML Safe Loading

```python
import yaml
from yaml.loader import SafeLoader

with open("users.yaml") as file:
    config = yaml.load(file, Loader=SafeLoader)  # Prevent code execution
```

Data Validation

Numeric Coercion

```python
# Prevent string injection in numeric columns
df_aligned.loc[:, c] = pd.to_numeric(df_aligned[c], errors='coerce')
```

Index Alignment

```python
# Prevent misaligned predictions
out['Pred_Product_Titer_gL'] = pd.Series(preds, index=X_aligned.index)
```

Performance Optimizations

Memory Management

Chunked Data Display

```python
# Preview only first 100 rows
st.dataframe(out.head(100), use_container_width=True)

# Avoid loading entire dataset into memory at once
```

Figure Cleanup

```python
# Proper matplotlib figure management
fig, ax = plt.subplots()
# ... plotting code ...
st.pyplot(fig)
plt.close(fig)  # Free memory
```

Computational Efficiency

Vectorized Operations

```python
# Batch prediction (vectorized)
preds = clf.predict(X_aligned)
```

```
# Vectorized column operations
df["Glucose_Consumption_Rate"] = df["Glucose_gL"].diff()
```

Lazy Evaluation

```python
# Only compute when needed
if st.button("▶️ Run"):
    st.session_state.has_run = True

# Execute only when triggered
if st.session_state.get("has_run") and input_df is not None:
    # Expensive computations here
```

Caching Benefits

Cache Type	Use Case	Invalidation
`@st.cache_resource`	Models, ML pipelines	Manual/App restart
`@st.cache_data`	DataFrames, JSON	Hash-based
Session State	User data, flags	Session expiry

Deployment Guide

Environment Setup

Requirements File

```txt
# requirements.txt
streamlit==1.28.0
scikit-learn==1.3.0
pandas==2.0.0
numpy==1.24.0
matplotlib==3.7.0
seaborn==0.12.0
joblib==1.2.0
streamlit-authenticator==0.2.0
PyYAML==6.0
```

Directory Structure

```text
bioreactor-dashboard/
├── app_streamlit.py
├── users.yaml
├── requirements.txt
├── outputs/
│   ├── model_ridgecv.joblib
│   ├── model_gbr.joblib
│   ├── model_linear.joblib
│   └── feature_schema.json
├── assets/
│   └── biotech_bg.png
└── sample_data/
    └── bioreactor_ml_dataset.csv
```

Configuration Files

users.yaml Template

```yaml
cookie:
  expiry_days: 30
  key: generate_secure_random_key_here
  name: bioreactor_dashboard_cookie
credentials:
  usernames:
    admin:
      email: admin@company.com
      name: Administrator
      password: $2b$12$hashed_password_here
      approved: true
      roles:
      - admin
      - user
```

feature_schema.json Template

```json
{
    "target": "Product_Titer_gL",
    "features": [
        "Temperature_C",
        "pH",
        "Glucose_gL",
        "Dissolved_Oxygen_percent",
        "Agitation_RPM",
        "Cell_Viability_percent",
        "Time_hours"
    ],
```

```
    "dtypes": {
        "Temperature_C": "float64",
        "pH": "float64",
        "Glucose_gL": "float64"
    },
    "trained_rows": 1000,
    "trained_cols": 7
}
```

Deployment Commands

Development Server

```bash
# Run locally
streamlit run app_streamlit.py

# With specific port
streamlit run app_streamlit.py --server.port 8501

# With network access
streamlit run app_streamlit.py --server.address 0.0.0.0
```

Production Deployment (Docker)

```dockerfile
# Dockerfile
FROM python:3.9-slim

WORKDIR /app

COPY requirements.txt .
RUN pip install -r requirements.txt

COPY . .

EXPOSE 8501

CMD ["streamlit", "run", "app_streamlit.py", "--server.port=8501", "--server.addr
ess=0.0.0.0"]
```

Docker Build & Run

```bash
# Build image
docker build -t bioreactor-dashboard .
```

```
# Run container
docker run -p 8501:8501 bioreactor-dashboard
```

Cloud Deployment Options

Streamlit Cloud

```yaml
# .streamlit/config.toml
[server]
maxUploadSize = 200
enableCORS = false
enableXsrfProtection = true

[browser]
gatherUsageStats = false
```

AWS EC2 Deployment

```bash
# Update system
sudo apt-get update
sudo apt-get install python3-pip

# Install dependencies
pip3 install -r requirements.txt

# Run with nohup for background execution
nohup streamlit run app_streamlit.py --server.port 8501 --server.address 0.0.0.0
&
```

Code Reference

Complete Function Index

Authentication Functions

Function	Purpose	Location
`load_config()`	Load users.yaml configuration	Lines 85-87
`authenticator.login()`	Handle user login	Line 96
`authenticator.register_user()`	New user registration	Lines 172-175

Data Processing Functions

Function	Purpose	Location
`preprocess()`	Feature engineering pipeline	Lines 170-199
`align_columns()`	Feature alignment with schema	Lines 201-218
`read_csv()`	Cached CSV loading	Lines 162-168

Visualization Functions

Function	Purpose	Location
`set_design()`	Dynamic CSS styling	Lines 22-83
`plot_correlation_heatmap()`	Correlation matrix	Lines 210-217
`plot_time_series()`	Temporal trends	Lines 218-230
`plot_feature_importance()`	Model interpretability	Lines 231-252

Model Functions

Function	Purpose	Location
`load_model()`	Cached model loading	Lines 151-154
`load_schema()`	Cached schema loading	Lines 156-160
`compute_rmse()`	RMSE calculation	Lines 220-222

Critical Code Sections

1. Main Execution Guard

```python
if not st.session_state.get("authentication_status"):
    st.stop()  # Prevent unauthenticated access
```

2. Prediction Pipeline

```python
# Core prediction logic
proc_df = preprocess(input_df)
```

```
X_aligned, align_info = align_columns(proc_df, schema)
preds = clf.predict(X_aligned)
```

3. Model Training Pipeline

```python
# Complete ML pipeline
pipe = Pipeline([
    ('imputer', SimpleImputer(strategy='median')),
    ('scaler', StandardScaler()),
    ('model', RidgeCV(alphas=np.logspace(-3, 3, 13), cv=5))
])
```

Appendices

Appendix A: Error Codes and Handling

Error Scenario	Handling Strategy	User Message
Missing target column	Check existence, show error	`st.error("Target column not found")`
Invalid credentials	Authentication fails	`st.sidebar.error('Invalid credentials')`
Model loading failure	Try-except with traceback	`st.exception(e)`
Data type mismatch	Coerce to numeric with errors='coerce'	Warning in alignment details
Missing features	Add NaN columns	Info in expander

Appendix B: Configuration Reference

Environment Variables

```bash
# Optional configurations
STREAMLIT_SERVER_PORT=8501
STREAMLIT_SERVER_ADDRESS=0.0.0.0
STREAMLIT_BROWSER_GATHER_USAGE_STATS=false
```

Streamlit Config (.streamlit/config.toml)

```toml
[theme]
primaryColor = "#0054A0"
backgroundColor = "#FFFFFF"
secondaryBackgroundColor = "#F0F2F6"
textColor = "#262730"
font = "sans serif"

[server]
maxUploadSize = 200
enableCORS = false
enableXsrfProtection = true
```

Appendix C: Model Performance Metrics

Metric	Formula	Interpretation
R^2	1 - (SS_res / SS_tot)	Variance explained
MAE	Σ|y_true - y_pred|/n	Average error magnitude
RMSE	√(Σ(y_true - y_pred)²/n)	Root mean square error

Appendix D: Feature Engineering Details

Derived Feature	Formula	Biological Significance
Glucose_Consumption_Rate	Δ[Glucose]/Δt	Metabolic activity rate
DO_Change	Δ[DO]/Δt	Oxygen uptake rate
Specific_Productivity	[Product]/Viability	Productivity per viable cell
High_Titer_Flag	[Product] > 1 g/L	High productivity indicator
Low_Viability_Flag	Viability < 98%	Culture health warning

Appendix E: Troubleshooting Guide

Common Issues and Solutions

1. **"No module named 'streamlit_authenticator'"**

```bash
pip install streamlit-authenticator
```

2. **Model loading fails**
   - Check file path exists
   - Verify joblib file isn't corrupted
   - Ensure scikit-learn version compatibility
3. **Authentication not working**
   - Verify users.yaml format
   - Check cookie key is set
   - Clear browser cookies
4. **Visualization not displaying**
   - Check matplotlib backend
   - Ensure figure is properly closed
   - Verify data types are numeric

Version History

Version	Date	Changes
1.0.0	Initial	Basic prediction dashboard
1.1.0	Update	Added authentication system
1.2.0	Update	Training tab implementation
1.3.0	Update	Dark mode and screen recorder

Contact & Support

For issues or questions:

- GitHub Issues: [repository-url]/issues
- Documentation: [docs-url]
- Email: support@bioreactor-ml.com

Document generated: March 2026
Last updated with code version: 1.3.0

```text
```

```
## How to Create the PDF

You can convert this markdown to PDF using any of these methods:

### Method 1: Using Pandoc (Command Line)
```bash
# Install pandoc
sudo apt-get install pandoc texlive-xetex

# Convert to PDF
pandoc documentation.md -o Bioreactor_ML_Dashboard_Documentation.pdf --pdf-engine
=xelatex
```

Method 2: Using Typora (GUI)

1. Open Typora
2. Paste the markdown content
3. File → Export → PDF

Method 3: Online Converters

- Use websites like mdtopdf.com
- Paste the content and download PDF

Method 4: VS Code with Extensions

1. Install "Markdown PDF" extension
2. Right-click on markdown file
3. Select "Markdown PDF: Export (pdf)"

The document includes comprehensive technical documentation with:

- Complete code explanations
- Architecture diagrams (ASCII)
- Function references with line numbers
- Configuration examples
- Deployment guides
- Troubleshooting sections
- Version history

This documentation should serve as both a user guide and developer reference for the Bioreactor ML Dashboard.
```